

Manvir Singh Cheema
Student Number: 218033894

Table of Contents

Part 1.....	3
1.1.....	4
1.2.....	6
1.3.....	7
1.4.....	8
Part 2.....	9
How I constructed world, what the imagery looked like.....	10
What imagery looked like.....	12
Working with the original (pre-implemented setup) model.....	13
Performance.....	15
What made it crash.....	18
Modifying the environment.....	19
Testing original CNN with modified environment.....	20
First architecture change (adapted convolutional layers) testing.....	25
Second architecture change (with dropout layers) testing.....	30
Recap.....	36
Supplementary Code.....	37

Part 1

1.1

A planar Robot can be represented by a 4x4 matrix as follows:

$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ 1 \end{bmatrix} = \begin{bmatrix} r_{1,1} & r_{1,2} & r_{1,3} & t_x \\ r_{2,1} & r_{2,2} & r_{2,3} & t_y \\ r_{3,1} & r_{3,2} & r_{3,3} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ 1 \end{bmatrix}$$

In order to represent the planar robot in a 4x4 matrix we can use DH (Denavit-Hartenberg) Link. The following are examples of the DH link frames.

Denavit-Hartenberg link frames

$$\text{Rot}(z_{n-1}, \theta_n) = \begin{pmatrix} \cos \theta_n & -\sin \theta_n & 0 & 0 \\ \sin \theta_n & \cos \theta_n & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad \text{Trans}(z_{n-1}, l_n) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & l_n \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$\text{Trans}(x_n, d_n) = \begin{pmatrix} 1 & 0 & 0 & d_n \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad \text{Rot}(x_n, \alpha_n) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \alpha_n & -\sin \alpha_n & 0 \\ 0 & \sin \alpha_n & \cos \alpha_n & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$${}_{n-1}T_n = \begin{pmatrix} \cos \theta_n & -\sin \theta_n \cos \alpha_n & \sin \theta_n \sin \alpha_n & d_n \cos \theta_n \\ \sin \theta_n & \cos \theta_n \cos \alpha_n & -\cos \theta_n \sin \alpha_n & d_n \sin \theta_n \\ 0 & \sin \alpha_n & \cos \alpha_n & l_n \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

To represent this specific planar robot setup we can first start by representing the prismatic joint's adjustable length l_2 and to the fixed length l_1 . To do this we replace the l_n within the DH link frame with $l_1 + l_2$, resulting in the following frame 1 (T1):

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & l_1 + l_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Next, to represent frame 2 we will perform a $\text{Rot}(z_1, \theta_1)$ dot product with $\text{Trans}(x_2, l_3)$, which will result in the following frame 2 (T2):

$$\begin{bmatrix} \cos \theta_1 & -\sin \theta_1 & 0 & l_3 \cos \theta_1 \\ \sin \theta_1 & \cos \theta_1 & 0 & l_3 \sin \theta_1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Finally, performing the dot product of T1 and T2 ($T_1 \cdot T_2$) will give is the final transformation matrix as follows:

$$\begin{bmatrix} \cos \theta_1 & -\sin \theta_1 & 0 & l_3 \cos \theta_1 \\ \sin \theta_1 & \cos \theta_1 & 0 & l_1 + l_2 + l_3 \sin \theta_1 \end{bmatrix}$$

	0	0	1	0	
	0	0	0	1	

1.2

A planar Robot can be represented by a 4x4 matrix as follows:

$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ 1 \end{bmatrix} = \begin{bmatrix} r_{1,1} & r_{1,2} & r_{1,3} & t_x \\ r_{2,1} & r_{2,2} & r_{2,3} & t_y \\ r_{3,1} & r_{3,2} & r_{3,3} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ 1 \end{bmatrix}$$

In order to represent the planar robot in a 4x4 matrix we can use DH (Denavit-Hartenberg) Link. To represent this specific planar robot setup in frame 1 we can first start by representing the first revolute joint that rotates the first link l_1 by θ_1 as $\text{Rot}(z_0, \theta_1)$. Along with this we can represent the translation along x_1 as $\text{Trans}(x_1, l_1)$. Finally for frame 1 (T1), this will simplify to

$$\begin{bmatrix} \cos \theta_1 & -\sin \theta_1 & 0 & l_1 \cos \theta_1 \\ \sin \theta_1 & \cos \theta_1 & 0 & l_1 \sin \theta_1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Next, to represent frame 2 we will perform a $\text{Rot}(z_1, \theta_2)$ dot product with $\text{Trans}(x_2, l_2)$, which will result in the following frame 2 (T2):

$$\begin{bmatrix} \cos \theta_2 & -\sin \theta_2 & 0 & l_2 \cos \theta_2 \\ \sin \theta_2 & \cos \theta_2 & 0 & l_2 \sin \theta_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Next, in order to represent frame 3 (T3), we will perform a dot product of rotation of $\text{Rot}(z_2, \theta_3)$ and translation of $\text{Trans}(x_3, l_3)$ resulting in the following:

$$\begin{bmatrix} \cos \theta_3 & -\sin \theta_3 & 0 & l_3 \cos \theta_3 \\ \sin \theta_3 & \cos \theta_3 & 0 & l_3 \sin \theta_3 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Finally, performing the dot product of T1, T2 and T3 ($T_1 \cdot T_2 \cdot T_3$) will give is the final transformation matrix as follows:

$$\begin{bmatrix} \cos \theta_1 \cos \theta_2 \cos \theta_3 & -(\sin \theta_1 \sin \theta_2 \sin \theta_3) & 0 & l_1 \cos \theta_1 * l_2 \cos \theta_2 * l_3 \cos \theta_3 \\ \sin \theta_1 \sin \theta_2 \sin \theta_3 & \cos \theta_1 \cos \theta_2 \cos \theta_3 & 0 & l_1 \sin \theta_1 * l_2 \sin \theta_2 * l_3 \sin \theta_3 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

1.3

A planar robot can be represented by a 4x4 matrix as follows:

$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ 1 \end{bmatrix} = \begin{bmatrix} r_{1,1} & r_{1,2} & r_{1,3} & t_x \\ r_{2,1} & r_{2,2} & r_{2,3} & t_y \\ r_{3,1} & r_{3,2} & r_{3,3} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ 1 \end{bmatrix}$$

In order to represent the planar robot in a 4x4 matrix we can use DH (Denavit-Hartenberg) Link. To represent this specific planar robot setup in frame 1 we can first start by representing the first revolute joint that rotates the first link l1 by θ_1 as $\text{Rot}(z_0, \theta_1)$. Along with this we can represent the translation along x1 as $\text{Trans}(x_1, l1)$. Finally for frame 1 (T1), this will simplify to

$$\begin{bmatrix} \cos \theta_1 & -\sin \theta_1 & 0 & l1 \cos \theta_1 \\ \sin \theta_1 & \cos \theta_1 & 0 & l1 \sin \theta_1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Next, to represent the prismatic joint frame we will perform a translation of z1 as $\text{Trans}(z_1, l2)$, which will result in the following frame 2 (T2):

$$\begin{bmatrix} 1 & 0 & 0 & l2 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Now, in order to represent frame 3 (T3), we will perform a dot product of rotation of $\text{Rot}(z_2, \theta_2)$ resulting in the following:

$$\begin{bmatrix} \cos \theta_2 & -\sin \theta_2 & 0 & l3 \cos \theta_2 \\ \sin \theta_2 & \cos \theta_2 & 0 & l3 \sin \theta_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Finally, performing the dot product of T1, T2 and T3 ($T1 \cdot T2 \cdot T3$) will give is the final transformation matrix as follows:

$$\begin{bmatrix} \cos \theta_1 + \cos \theta_2 & -(\sin \theta_1 + \sin \theta_2) & 0 & l1 \cos \theta_1 + l2 \cos \theta_1 + l3 \cos (\theta_1 + \theta_2) \\ \sin \theta_1 + \sin \theta_2 & \cos \theta_1 + \cos \theta_2 & 0 & l1 \sin \theta_1 + l2 \sin \theta_1 + l3 \sin (\theta_1 + \theta_2) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

1.4

To represent this specific non-planar robot setup in frame 1 we can first start by representing the first revolute joint that rotates the first link l_1 by θ_1 as $\text{Rot}(z_0, \theta_1)$. Along with this we can represent the translation along x_1 as $\text{Trans}(x_1, l_1)$. Finally for frame 1 (T_1), this will simplify to

$$\begin{bmatrix} \cos \theta_1 & -\sin \theta_1 & 0 & l_1 \cos \theta_1 \\ \sin \theta_1 & \cos \theta_1 & 0 & l_1 \sin \theta_1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Next, to represent the second revolute joint frame we will perform a rotation to the second link by θ_2 as $\text{Rot}(z_1, \theta_2)$. Along with this we can represent the translation along x_2 as $\text{Trans}(x_2, l_2)$. Finally this will simplify to the following frame 2 (T_2):

$$\begin{bmatrix} \cos \theta_2 & -\sin \theta_2 & 0 & l_2 \cos \theta_2 \\ \sin \theta_2 & \cos \theta_2 & 0 & l_2 \sin \theta_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

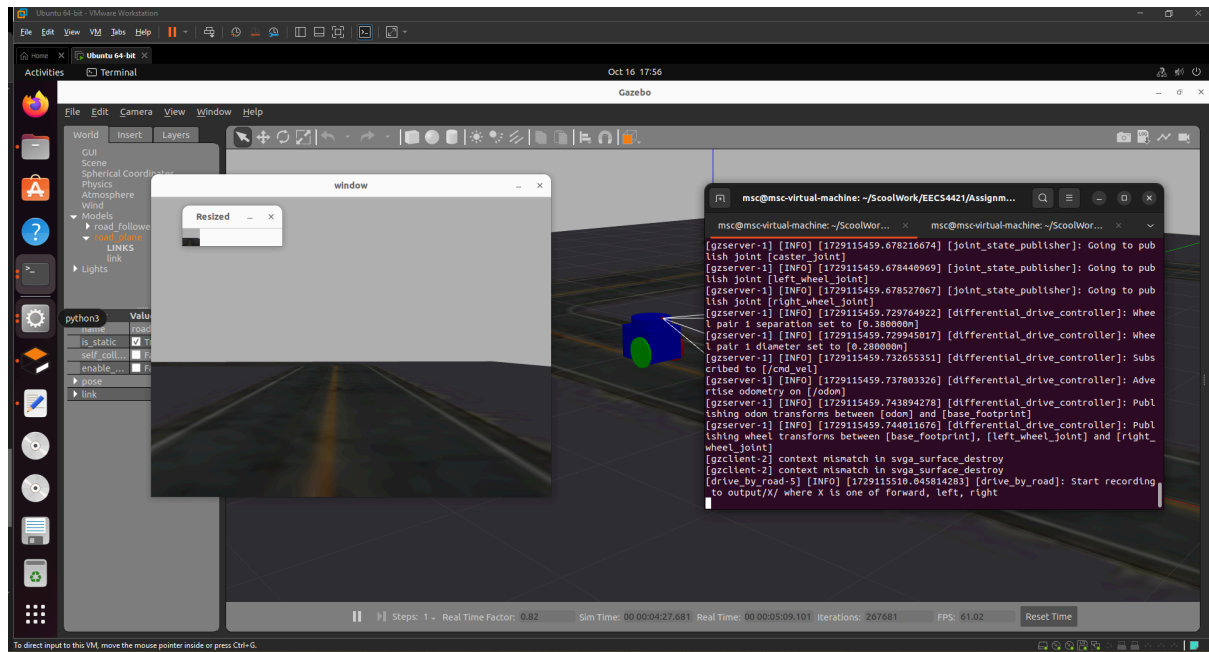
Finally, performing the dot product of T_1 and T_2 ($T_1 \cdot T_2$) will give is the final transformation matrix as follows:

$$\begin{bmatrix} \cos \theta_1 + \cos \theta_2 & -(\sin \theta_1 + \sin \theta_2) & 0 & l_1 \cos \theta_1 + l_2 \cos (\theta_1 + \theta_2) \\ \sin \theta_1 + \sin \theta_2 & \cos \theta_1 + \cos \theta_2 & 0 & l_1 \sin \theta_1 + l_2 \sin (\theta_1 + \theta_2) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

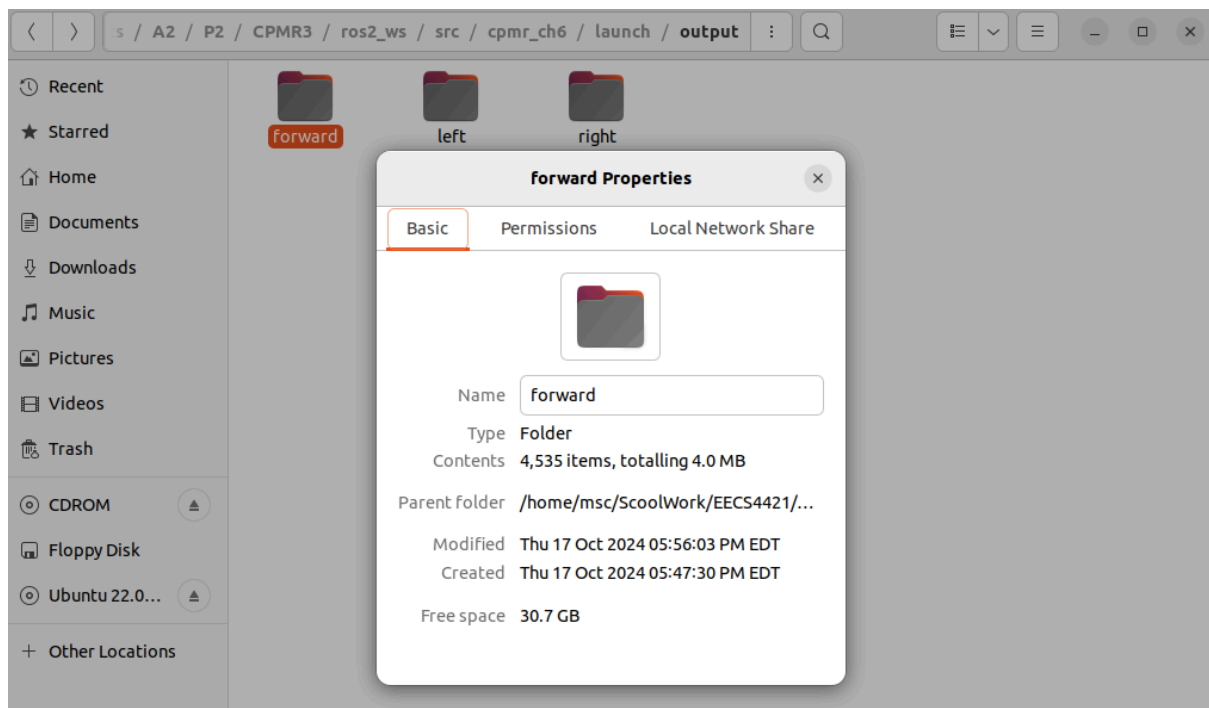
Part 2

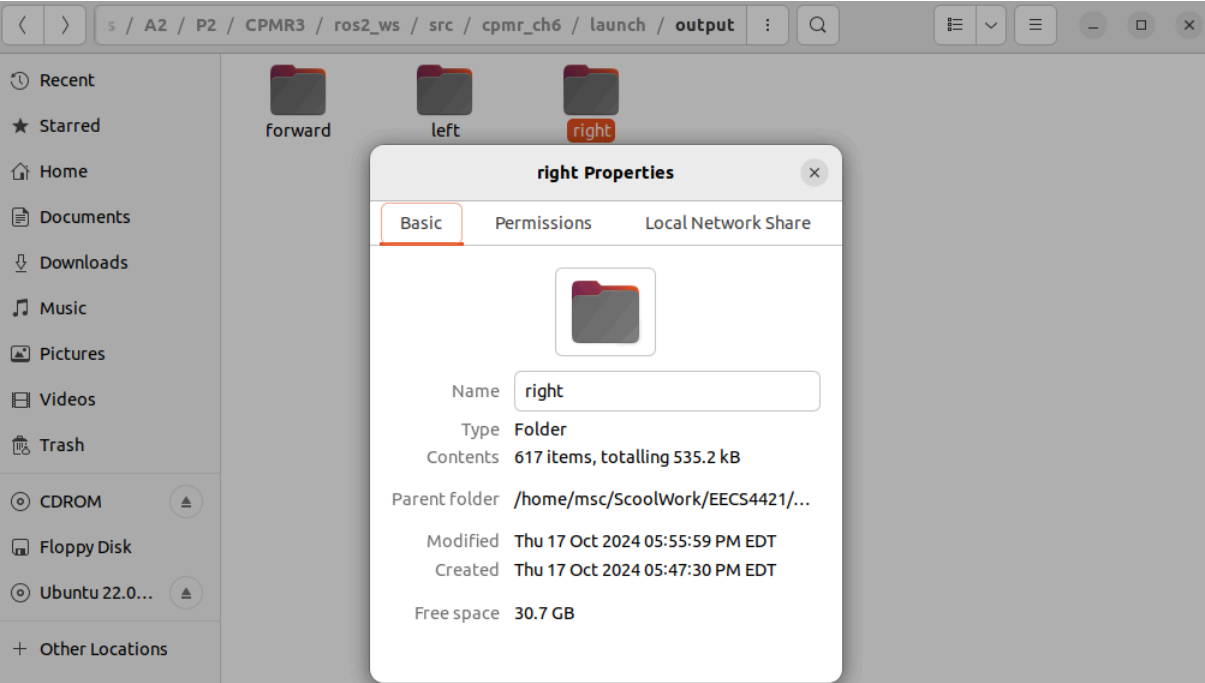
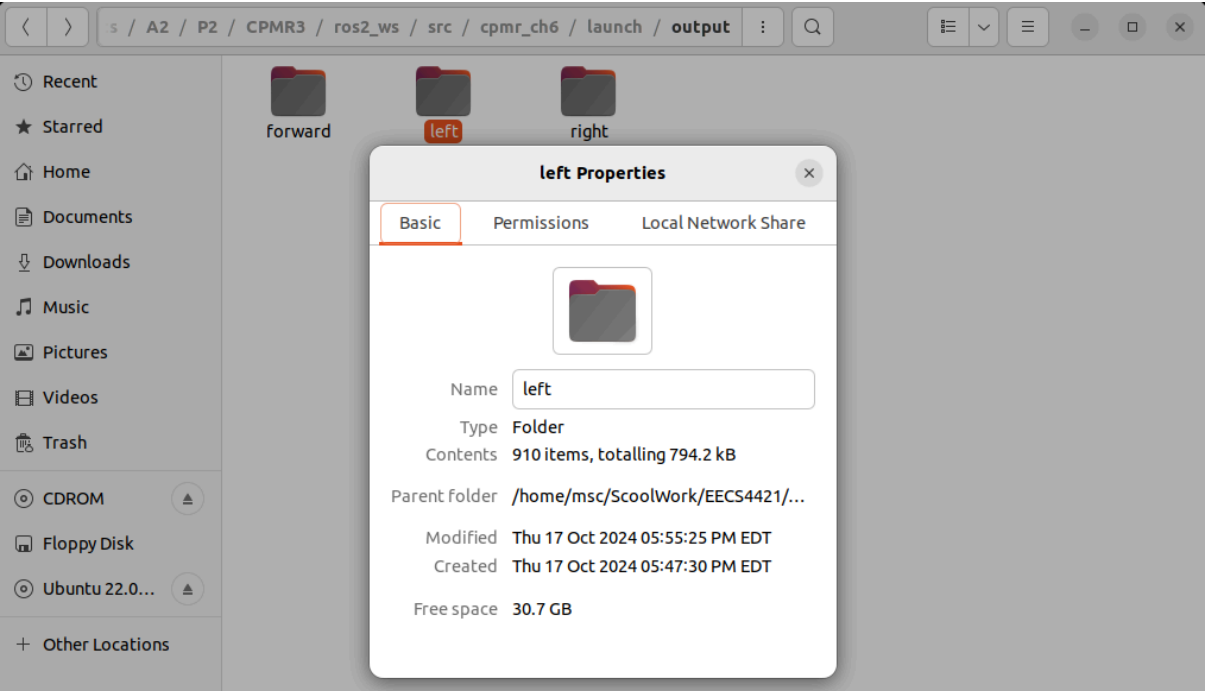
How I constructed world, what the imagery looked like

Collecting Data:



Collected Data:

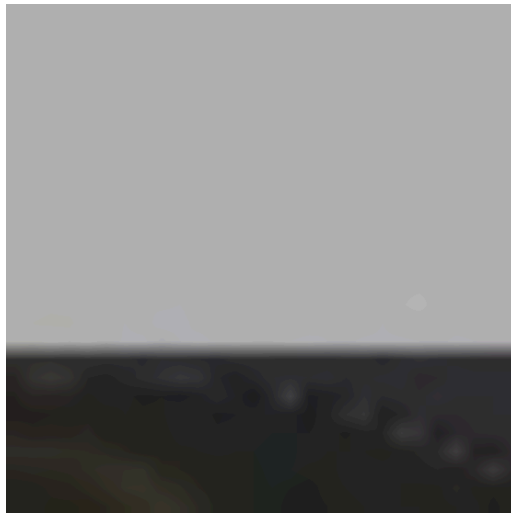




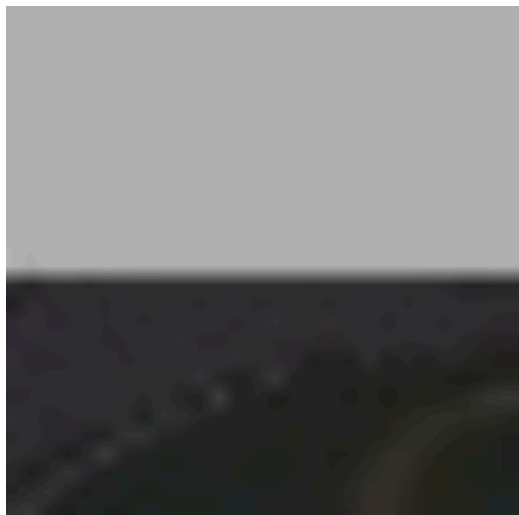
What imagery looked like

Example Data:

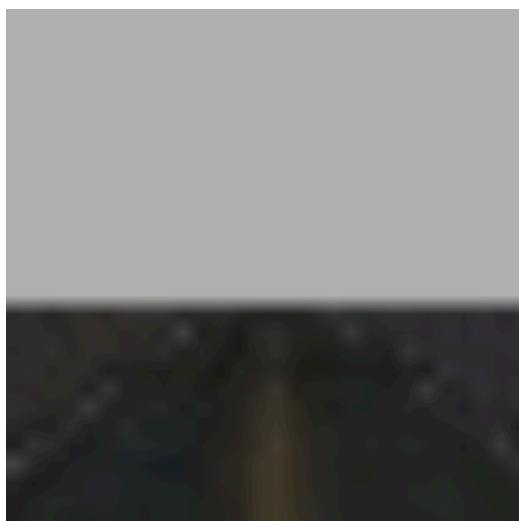
→ Left



→ Right



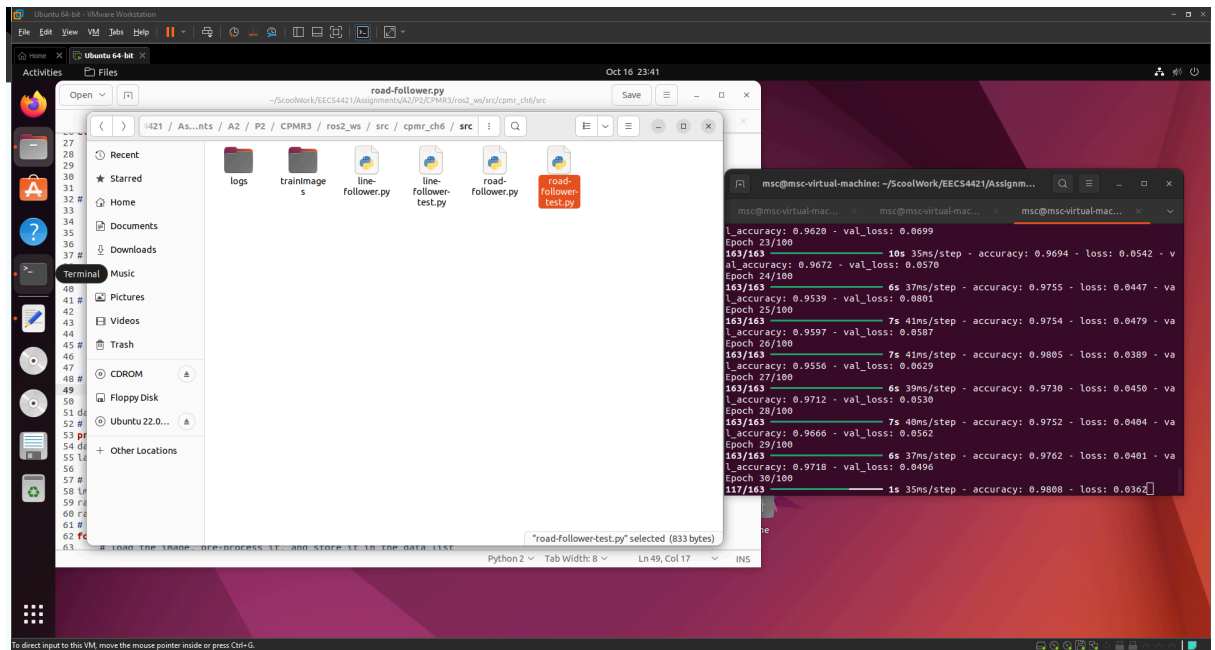
→ Forward



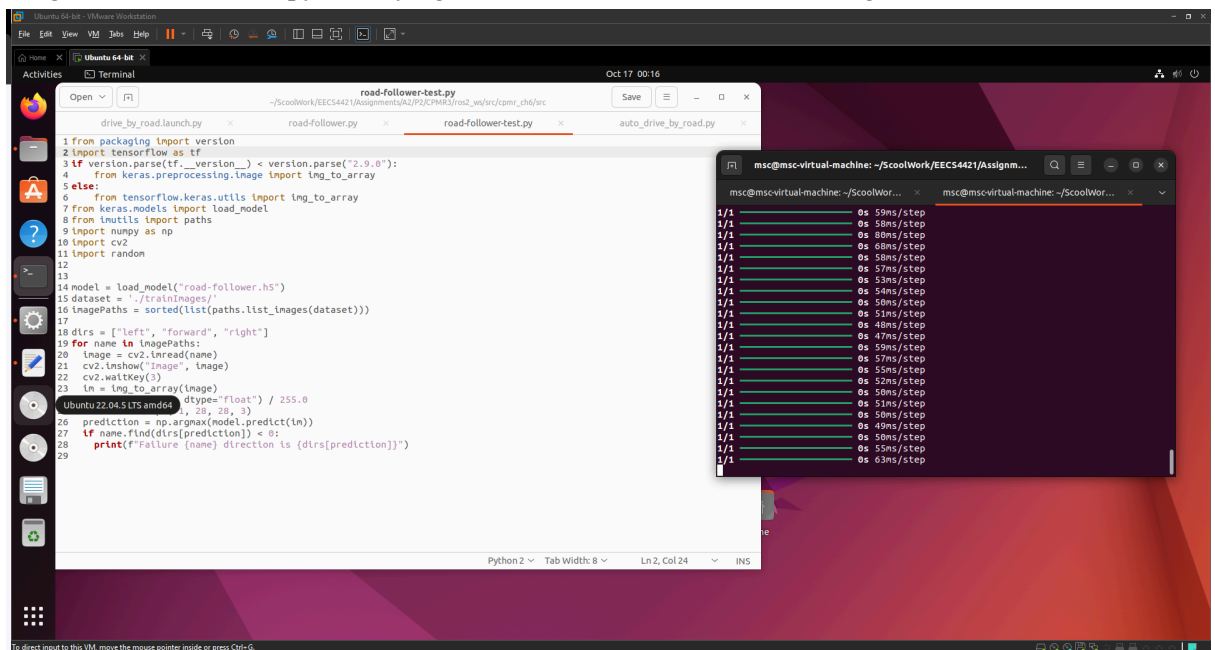
Working with the original (pre-implemented setup) model

Training and creating original model using road-follower:

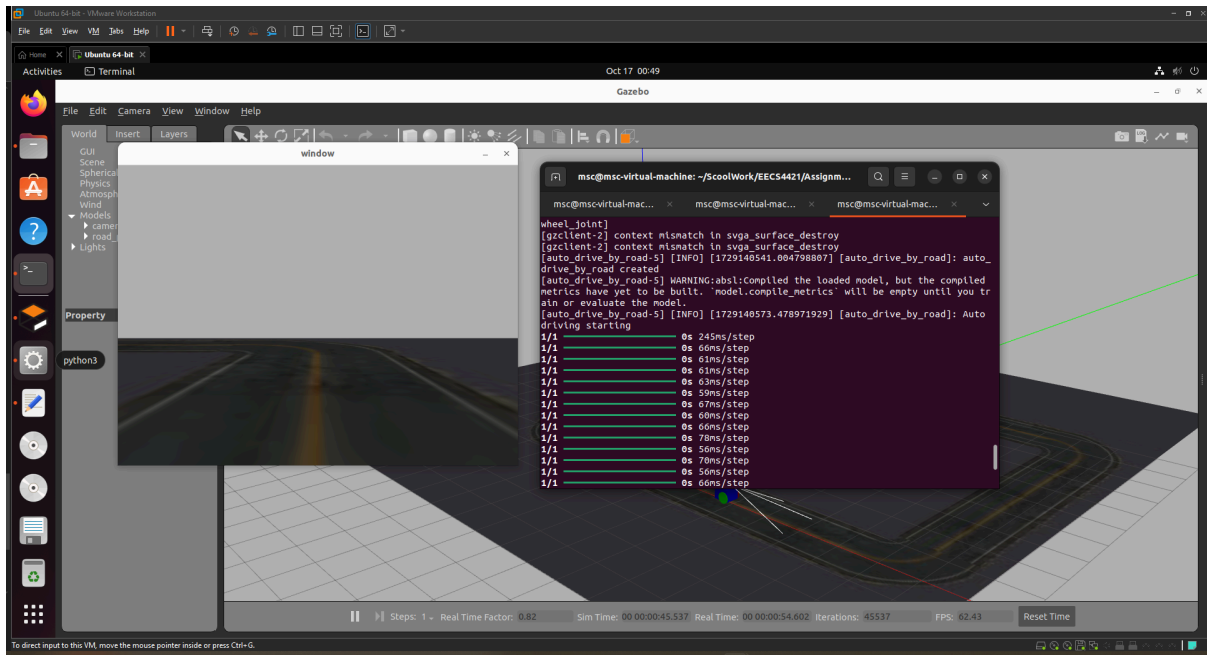
- Downloaded all required things and then copied the output folder to the directory and renamed it to trainImages
- Then I changed lr to learning rate and changed the import; from tensorflow.keras.layers instead of keras.layers.convolutional



Testing using road-follower-test.py modifying the model name with correct ending:

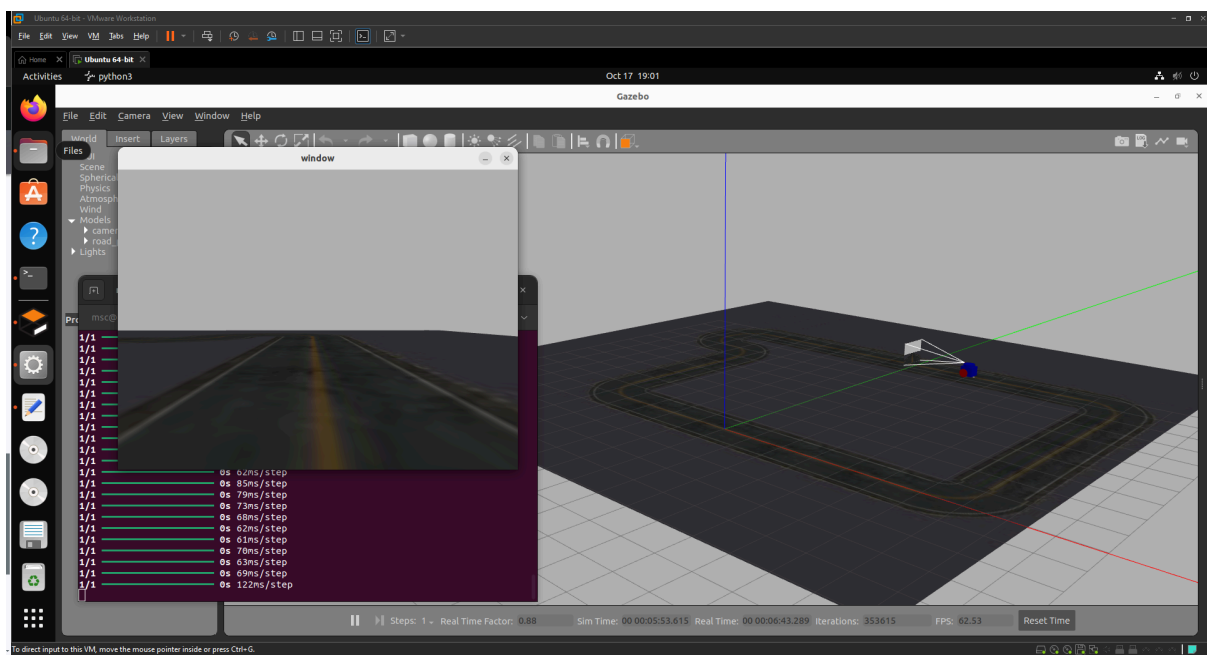
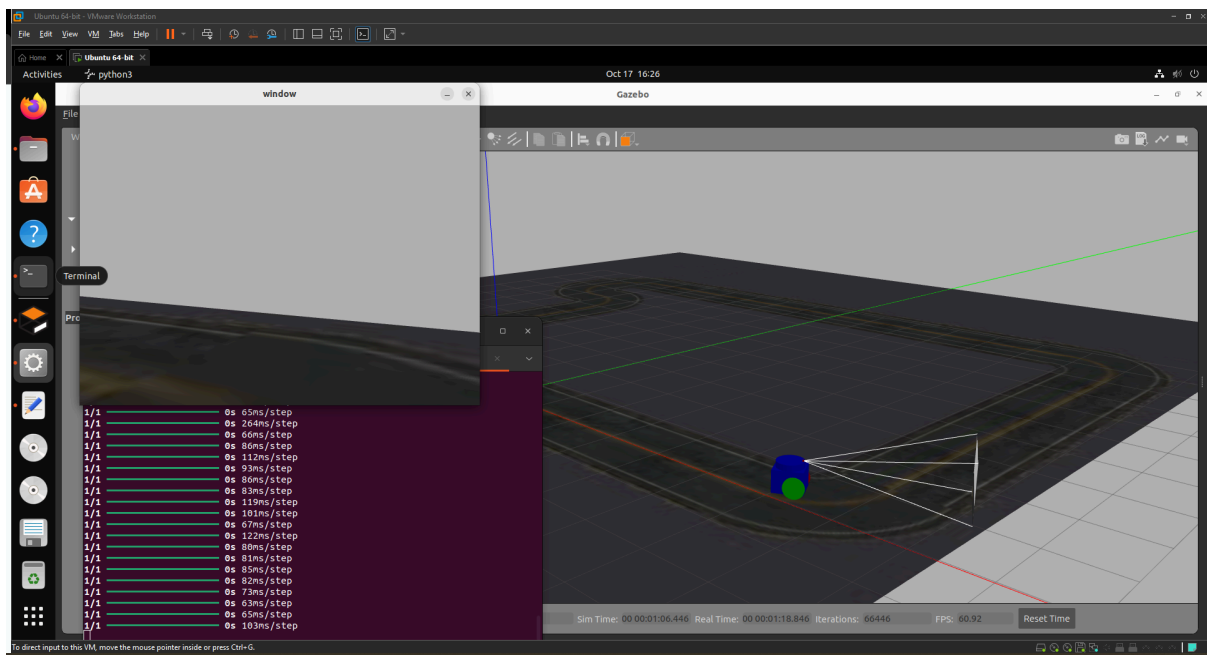


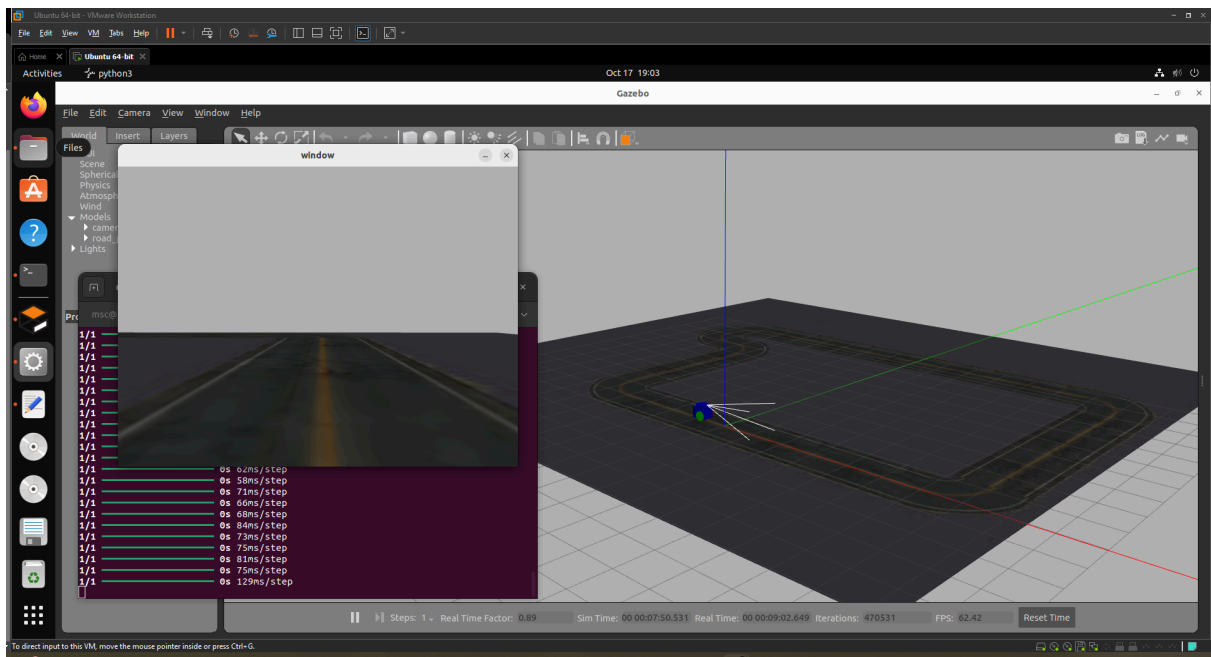
Using `auto_drive_by_road.launch.py` with model:



Performance

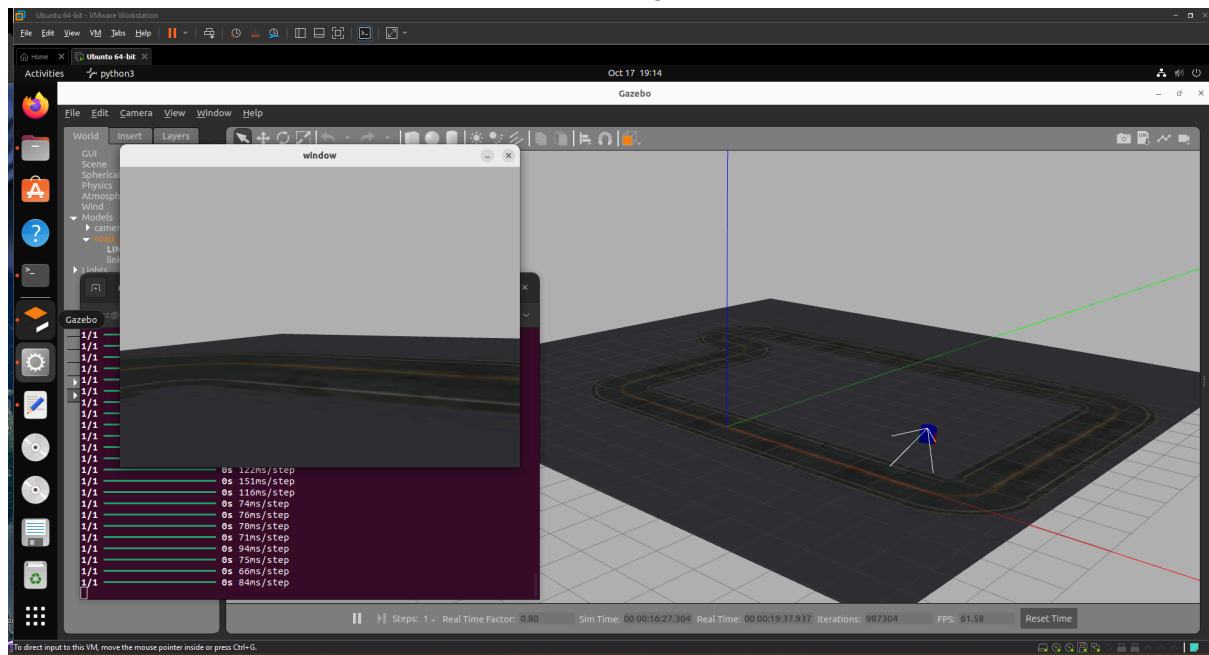
Successfully works and completes the lap. Able to manoeuvre the curves





What made it crash

However, on the 3rd round it fails and is unable to navigate back:



Modifying the environment

Adding objects and testing ability of basic trained NN:

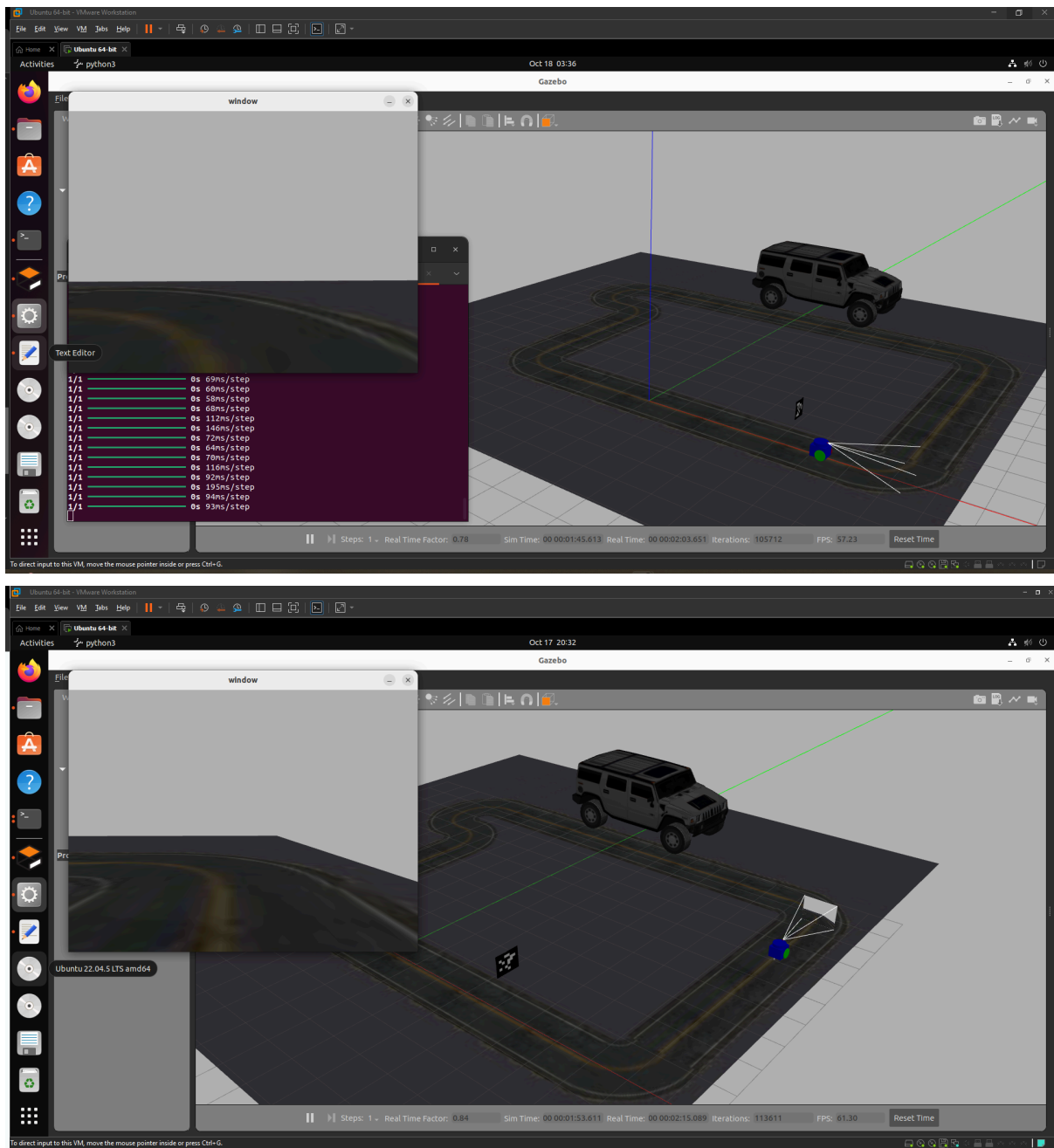
Added:

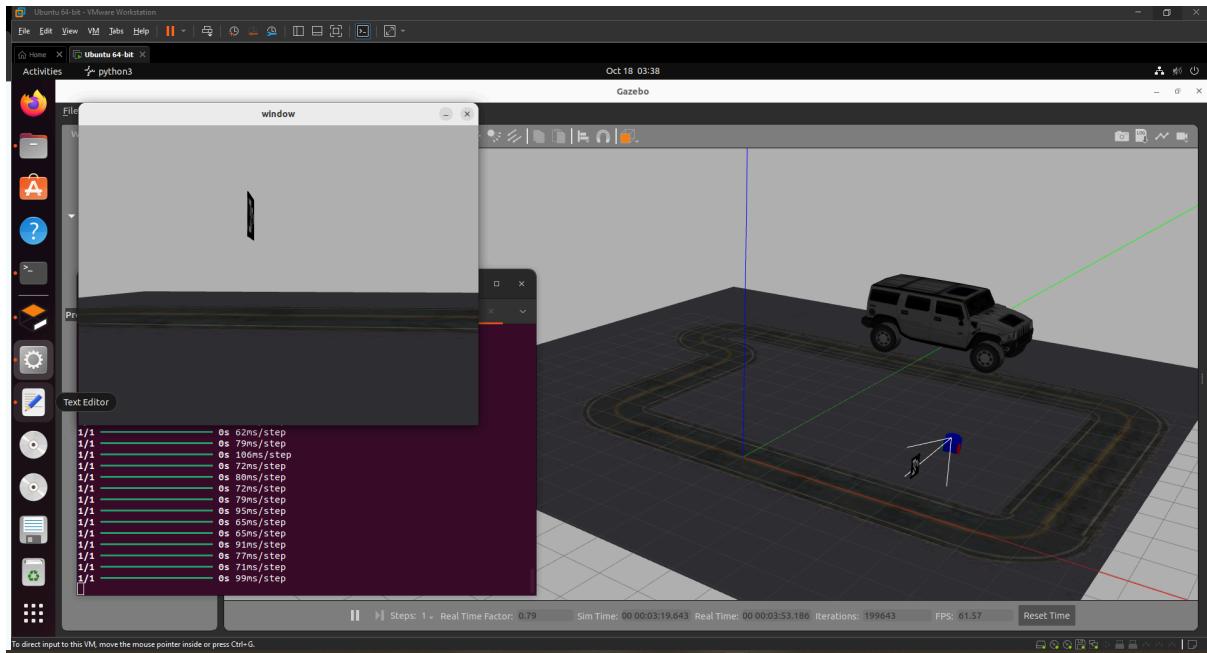
- Hummer hovering over track
- Aruco

Testing original CNN with modified environment

Testing:

Robot fails when trained on original model images





For the original model (without architecture modifications):

How well does the model work and how sensitive is it to other things in the world?

I believe it is more sensitive as whenever it crosses near the Aruco it creates a notification that is visible in the terminal which can possibly lead to errors in navigation and eventually failing and nevering getting back on track

How well did your network generalise to this novel environment?

Not quite well, it definitely did not perform as well as compared to when no objects were present within the environment, since it was unable to even complete 1 lap, maybe due to confusion within interpreting the environment with the new objects, but still successfully completed track. There were some over corrections made by the NN visible when near objects specifically the SUV that caused it to veer off.

Original road-follower.py:

```
#
# This network is based on the Line Follower Robot using CNN by Nawaz Ahmad
# towardsdatascience.com
#
from packaging import version
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Activation, Flatten, Dense
from tensorflow.keras.preprocessing.image import ImageDataGenerator
if version.parse(tf.__version__) < version.parse("2.9.0"):
    from keras.preprocessing.image import img_to_array
else:
    from tensorflow.keras.utils import img_to_array
from tensorflow.keras.optimizers import Adam
from sklearn.model_selection import train_test_split
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.callbacks import TensorBoard
```

```

from imutils import paths
import numpy as np
import argparse
import random
import cv2
import os
from datetime import datetime

class LeNet:
    @staticmethod
    def build(width, height, depth, classes):
        # initialize the model
        model = Sequential()
        inputShape = (height, width, depth)
        # first set of CONV => RELU => POOL layers
        model.add(Conv2D(20, (5, 5), padding="same",
            input_shape=inputShape))
        model.add(Activation("relu"))
        model.add(MaxPooling2D(pool_size=(2, 2), strides=(2, 2)))
        # second set of CONV => RELU => POOL layers
        model.add(Conv2D(50, (5, 5), padding="same"))
        model.add(Activation("relu"))
        model.add(MaxPooling2D(pool_size=(2, 2), strides=(2, 2)))
        # first (and only) set of FC => RELU layers
        model.add(Flatten())
        model.add(Dense(500))
        model.add(Activation("relu"))
        # softmax classifier
        model.add(Dense(classes))
        model.add(Activation("softmax"))
        # return the constructed network architecture
        return model

dataset = './trainImages/'
# initialize the data and labels
print("[INFO] loading images...")
data = []
labels = []

# grab the image paths and randomly shuffle them
imagePaths = sorted(list(paths.list_images(dataset)))
random.seed(42)
random.shuffle(imagePaths)
# loop over the input images
for imagePath in imagePaths:
    # load the image, pre-process it, and store it in the data list
    image = cv2.imread(imagePath)
    image = cv2.resize(image, (28, 28))

```

```

    image = img_to_array(image)
    data.append(image)
# extract the class label from the image path and update the
# labels list
    label = imagePath.split(os.path.sep)[-2]
    print(label)
    if label == 'left':
        label = 0
    elif label == 'forward':
        label = 1
    else:
        label = 2
    labels.append(label)
# scale the raw pixel intensities to the range [0, 1]
data = np.array(data, dtype="float") / 255.0
labels = np.array(labels)

# partition the data into training and testing splits using 75% of
# the data for training and the remaining 25% for testing
(trainX, testX, trainY, testY) = train_test_split(data,
    labels, test_size=0.25, random_state=42)
# convert the labels from integers to vectors
trainY = to_categorical(trainY, num_classes=3)
testY = to_categorical(testY, num_classes=3)

logdir = "logs/scalars/" + datetime.now().strftime("%Y%m%d-%H%M%S")
tensorboard_callback = TensorBoard(log_dir=logdir)

# initialize the number of epochs to train for, initial learning rate,
# and batch size
EPOCHS = 100
INIT_LR = 1e-3
BS = 32
# initialize the model
print("[INFO] compiling model...")
model = LeNet.build(width=28, height=28, depth=3, classes=3)
opt = Adam(learning_rate=INIT_LR, decay=INIT_LR / EPOCHS)
model.compile(loss="binary_crossentropy", optimizer=opt,
    metrics=["accuracy"])

# train the network
print("[INFO] training network...")
H = model.fit(trainX, trainY, batch_size=BS,
    validation_data=(testX, testY), # steps_per_epoch=len(trainX) // BS,
    epochs=EPOCHS, verbose=1,
    callbacks=[tensorboard_callback])

```

```
# save the model to disk
print("[INFO] serializing network...")
model.save("road-follower.keras")
```

First architecture change (adapted convolutional layers) testing

Changes made to the road-follower.py to create road-follower-v2.py:

1. Increased convolutional layers from 2 to 4.
2. Adjusted filters per layer: 32, 64, 128, and 256.
3. Reduced kernel size from (5, 5) to (3, 3) for all layers.
4. Added an additional fully connected (dense) layer with 512 units.

road-follower-v2.py code:

```
# This network is based on the Line Follower Robot using CNN by Nawaz Ahmad
# towardsdatascience.com
#
from packaging import version
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Activation, Flatten, Dense
from tensorflow.keras.preprocessing.image import ImageDataGenerator
if version.parse(tf.__version__) < version.parse("2.9.0"):
    from keras.preprocessing.image import img_to_array
else:
    from tensorflow.keras.utils import img_to_array
from tensorflow.keras.optimizers import Adam
from sklearn.model_selection import train_test_split
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.callbacks import TensorBoard
from imutils import paths
import numpy as np
import argparse
import random
import cv2
import os
from datetime import datetime

class LeNet:
    @staticmethod
    def build(width, height, depth, classes):
        # initialize the model
        model = Sequential()
        inputShape = (height, width, depth)

        # first set of CONV => RELU => POOL layers
        model.add(Conv2D(32, (3, 3), padding="same", input_shape=inputShape))
        model.add(Activation("relu"))
        model.add(MaxPooling2D(pool_size=(2, 2), strides=(2, 2)))

        # second set of CONV => RELU => POOL layers
        model.add(Conv2D(64, (3, 3), padding="same"))
```

```

model.add(Activation("relu"))
model.add(MaxPooling2D(pool_size=(2, 2), strides=(2, 2)))

# third set of CONV => RELU => POOL layers
model.add(Conv2D(128, (3, 3), padding="same"))
model.add(Activation("relu"))
model.add(MaxPooling2D(pool_size=(2, 2), strides=(2, 2)))

# fourth set of CONV => RELU => POOL layers
model.add(Conv2D(256, (3, 3), padding="same"))
model.add(Activation("relu"))
model.add(MaxPooling2D(pool_size=(2, 2), strides=(2, 2)))

# first set of FC => RELU layers
model.add(Flatten())
model.add(Dense(1024))
model.add(Activation("relu"))

# second set of FC => RELU layers
model.add(Dense(512))
model.add(Activation("relu"))

# softmax classifier
model.add(Dense(classes))
model.add(Activation("softmax"))

# return the constructed network architecture
return model

dataset = './trainImages/'
# initialize the data and labels
print("[INFO] loading images...")
data = []
labels = []

# grab the image paths and randomly shuffle them
imagePaths = sorted(list(paths.list_images(dataset)))
random.seed(42)
random.shuffle(imagePaths)

# loop over the input images
for imagePath in imagePaths:
    # load the image, pre-process it, and store it in the data list
    image = cv2.imread(imagePath)
    image = cv2.resize(image, (28, 28))
    image = img_to_array(image)
    data.append(image)

```

```

# extract the class label from the image path and update the labels list
label = imagePath.split(os.path.sep)[-2]
if label == 'left':
    label = 0
elif label == 'forward':
    label = 1
else:
    label = 2
labels.append(label)

# scale the raw pixel intensities to the range [0, 1]
data = np.array(data, dtype="float") / 255.0
labels = np.array(labels)

# partition the data into training and testing splits using 75% of
# the data for training and the remaining 25% for testing
(trainX, testX, trainY, testY) = train_test_split(data,
    labels, test_size=0.25, random_state=42)

# convert the labels from integers to vectors
trainY = to_categorical(trainY, num_classes=3)
testY = to_categorical(testY, num_classes=3)

logdir = "logs/scalars/" + datetime.now().strftime("%Y%m%d-%H%M%S")
tensorboard_callback = TensorBoard(log_dir=logdir)

# initialize the number of epochs to train for, initial learning rate, and batch size
EPOCHS = 100
INIT_LR = 1e-3
BS = 32

# initialize the model
print("[INFO] compiling model...")
model = LeNet.build(width=28, height=28, depth=3, classes=3)
opt = Adam(learning_rate=INIT_LR, decay=INIT_LR / EPOCHS)
model.compile(loss="binary_crossentropy", optimizer=opt, metrics=["accuracy"])

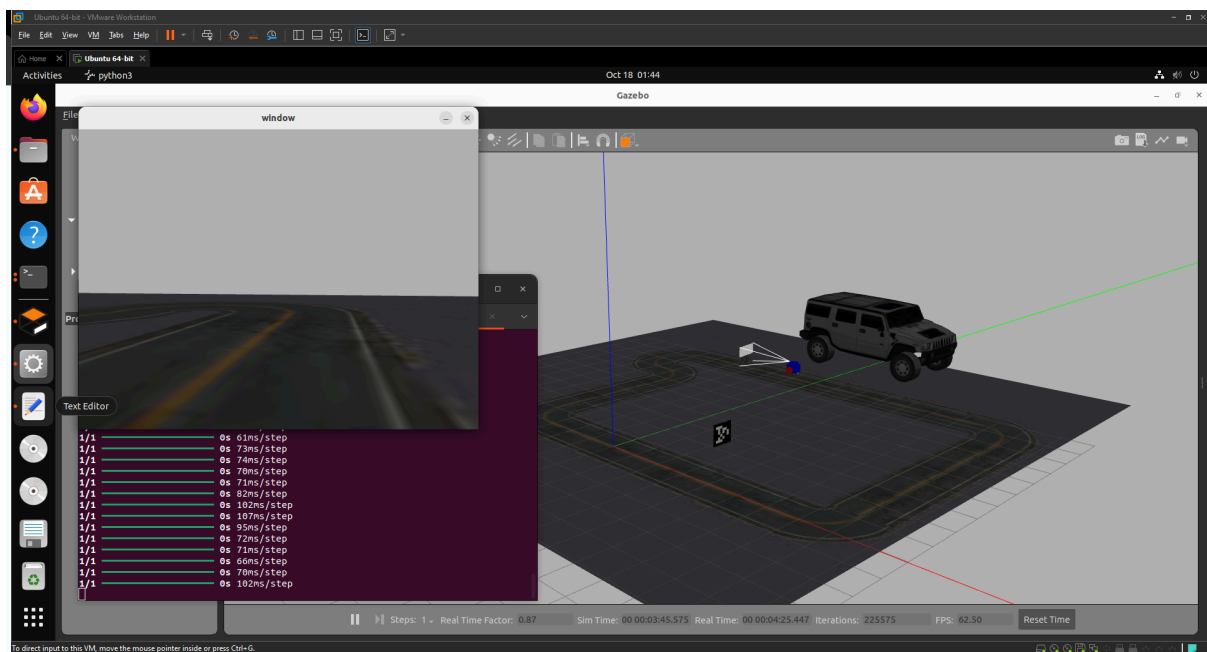
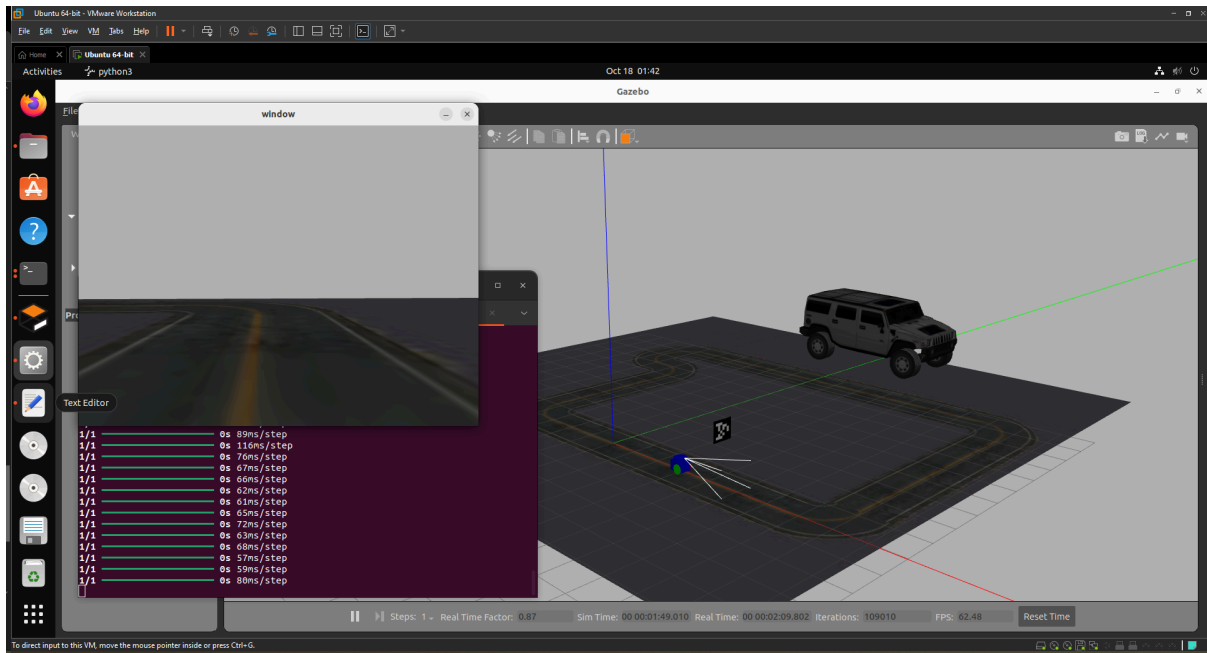
# train the network
print("[INFO] training network...")
H = model.fit(trainX, trainY, batch_size=BS,
    validation_data=(testX, testY),
    epochs=EPOCHS, verbose=1, callbacks=[tensorboard_callback])

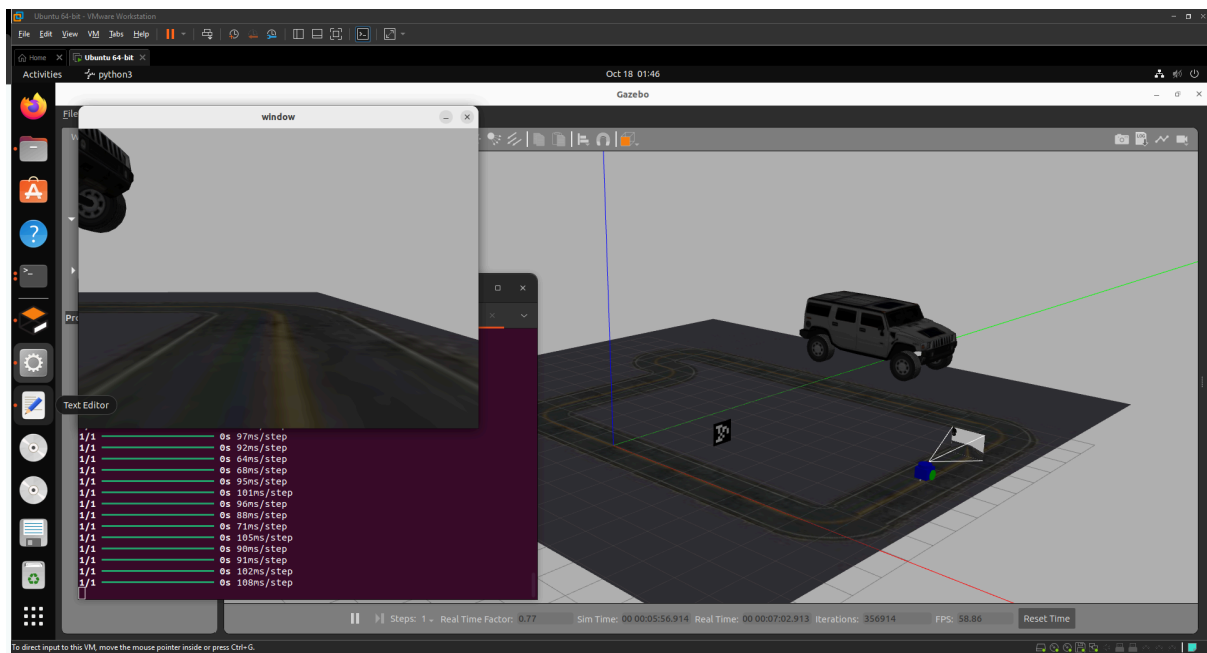
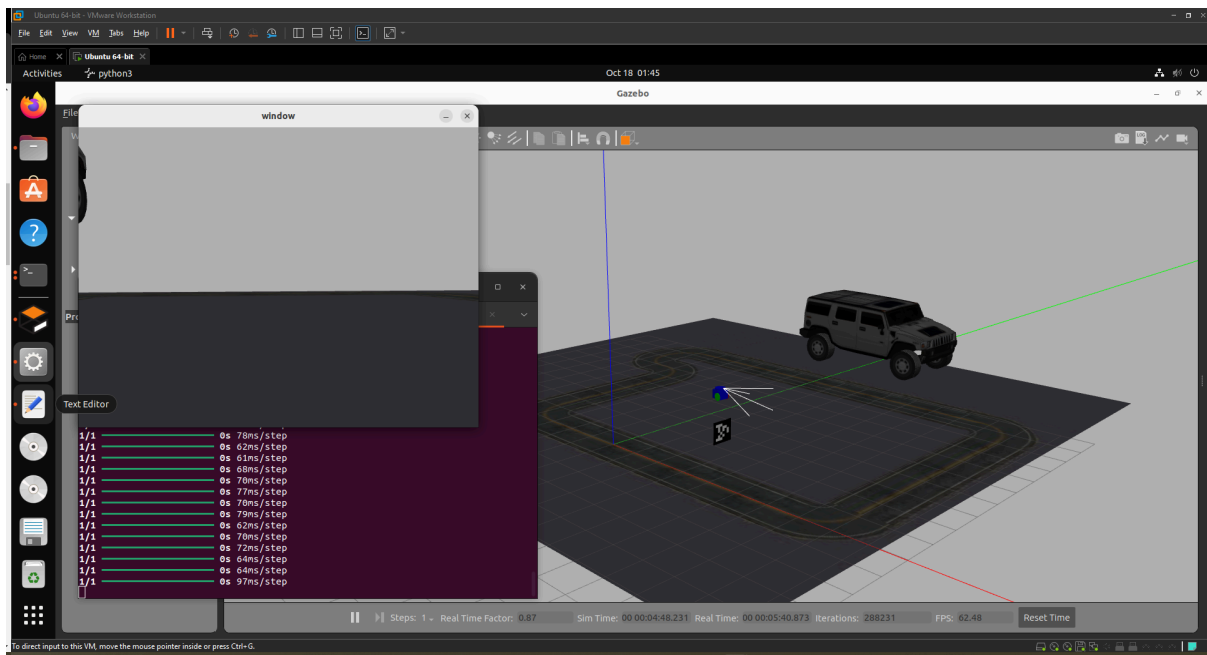
# save the model to disk
print("[INFO] serializing network...")
model.save("road-follower-updated.keras")

```

Results of road-follower-v2.py:

Fails in first lap, but successfully manoeuvres back onto track, and then again veers off repetitively.





road-follower-v2.py performance:
Overall worse than the original model

Second architecture change (with dropout layers) testing

Changes made to the road-follower.py to create road-follower-v3.py:

1. Dropout Layers: Added Dropout layers after the convolutional layers and the fully connected layer. Dropout helps prevent overfitting by randomly setting a fraction of input units to 0 during training.
 - a. Dropout(0.25) added after both pooling layers.
 - b. Dropout(0.5) added after the dense (fully connected) layer.
2. Loss Function: Changed the loss function to categorical_crossentropy because this is more appropriate for multiclass classification problems.

road-follower-v3.py code:

```
from packaging import version
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Activation, Flatten, Dense, Dropout
from tensorflow.keras.preprocessing.image import ImageDataGenerator
if version.parse(tf.__version__) < version.parse("2.9.0"):
    from keras.preprocessing.image import img_to_array
else:
    from tensorflow.keras.utils import img_to_array
from tensorflow.keras.optimizers import Adam
from sklearn.model_selection import train_test_split
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.callbacks import TensorBoard
from imutils import paths
import numpy as np
import argparse
import random
import cv2
import os
from datetime import datetime

class LeNet:
    @staticmethod
    def build(width, height, depth, classes):
        # initialize the model
        model = Sequential()
        inputShape = (height, width, depth)

        # first set of CONV => RELU => POOL layers
        model.add(Conv2D(20, (5, 5), padding="same", input_shape=inputShape))
        model.add(Activation("relu"))
        model.add(MaxPooling2D(pool_size=(2, 2), strides=(2, 2)))
        model.add(Dropout(0.25)) # Add dropout to avoid overfitting

        # second set of CONV => RELU => POOL layers
```

```

model.add(Conv2D(50, (5, 5), padding="same"))
model.add(Activation("relu"))
model.add(MaxPooling2D(pool_size=(2, 2), strides=(2, 2)))
model.add(Dropout(0.25)) # Add dropout

# first (and only) set of FC => RELU layers
model.add(Flatten())
model.add(Dense(500))
model.add(Activation("relu"))
model.add(Dropout(0.5)) # Add dropout to fully connected layer

# softmax classifier
model.add(Dense(classes))
model.add(Activation("softmax"))

# return the constructed network architecture
return model

dataset = './trainImages/'
# initialize the data and labels
print("[INFO] loading images...")
data = []
labels = []

# grab the image paths and randomly shuffle them
imagePaths = sorted(list(paths.list_images(dataset)))
random.seed(42)
random.shuffle(imagePaths)

# loop over the input images
for imagePath in imagePaths:
    # load the image, pre-process it, and store it in the data list
    image = cv2.imread(imagePath)
    image = cv2.resize(image, (28, 28))
    image = img_to_array(image)
    data.append(image)
    # extract the class label from the image path and update the labels list
    label = imagePath.split(os.path.sep)[-2]
    if label == 'left':
        label = 0
    elif label == 'forward':
        label = 1
    else:
        label = 2
    labels.append(label)

# scale the raw pixel intensities to the range [0, 1]
data = np.array(data, dtype="float") / 255.0

```

```

labels = np.array(labels)

# partition the data into training and testing splits using 75% of
# the data for training and the remaining 25% for testing
(trainX, testX, trainY, testY) = train_test_split(data, labels, test_size=0.25, random_state=42)

# convert the labels from integers to vectors
trainY = to_categorical(trainY, num_classes=3)
testY = to_categorical(testY, num_classes=3)

logdir = "logs/scalars/" + datetime.now().strftime("%Y%m%d-%H%M%S")
tensorboard_callback = TensorBoard(log_dir=logdir)

# initialize the number of epochs to train for, initial learning rate, and batch size
EPOCHS = 100
INIT_LR = 1e-3
BS = 32

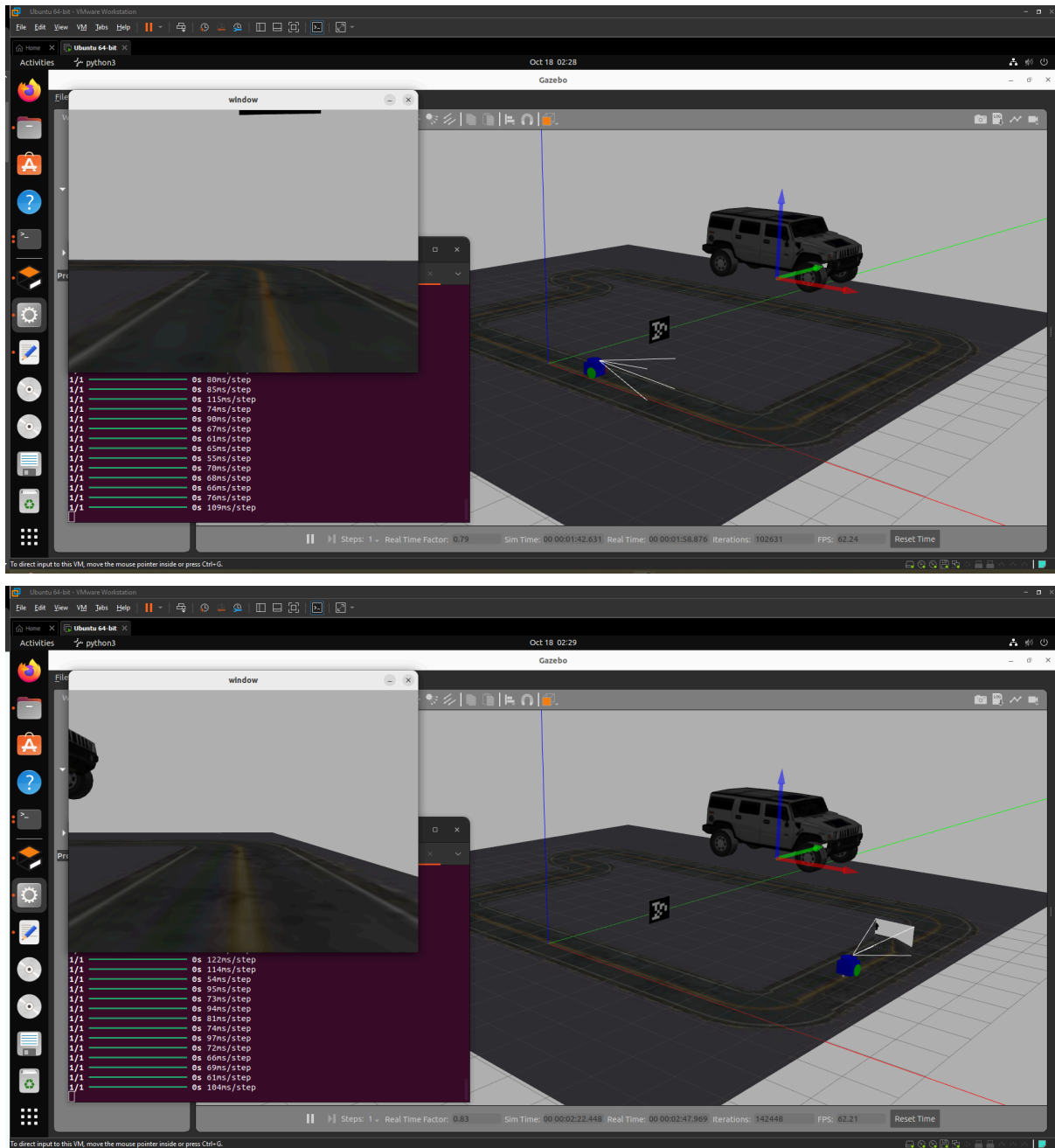
# initialize the model
print("[INFO] compiling model...")
model = LeNet.build(width=28, height=28, depth=3, classes=3)
opt = Adam(learning_rate=INIT_LR, decay=INIT_LR / EPOCHS)
model.compile(loss="categorical_crossentropy", optimizer=opt, metrics=["accuracy"])

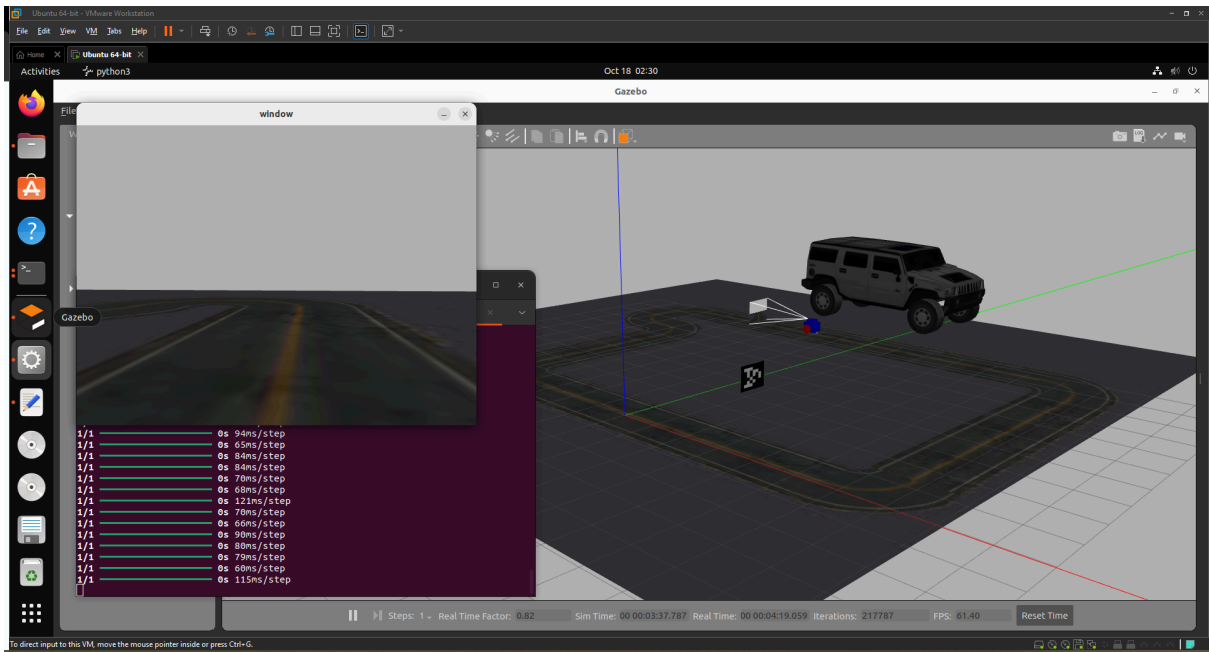
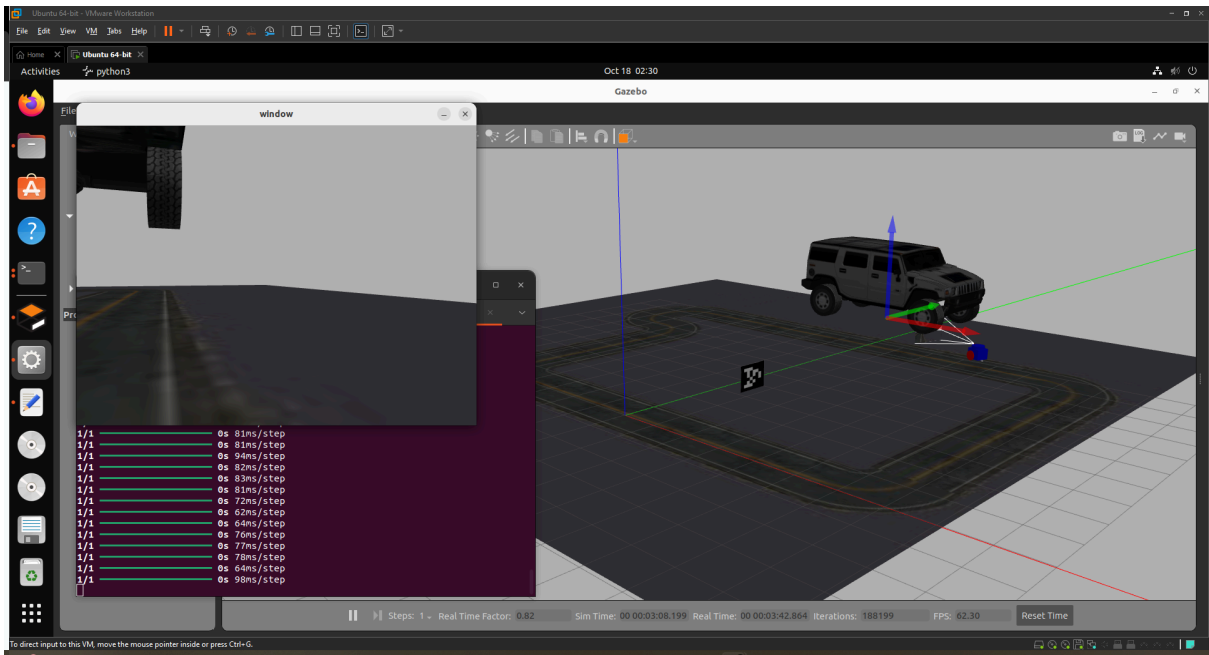
# train the network
print("[INFO] training network...")
H = model.fit(trainX, trainY, batch_size=BS, validation_data=(testX, testY), epochs=EPOCHS,
verbose=1, callbacks=[tensorboard_callback])

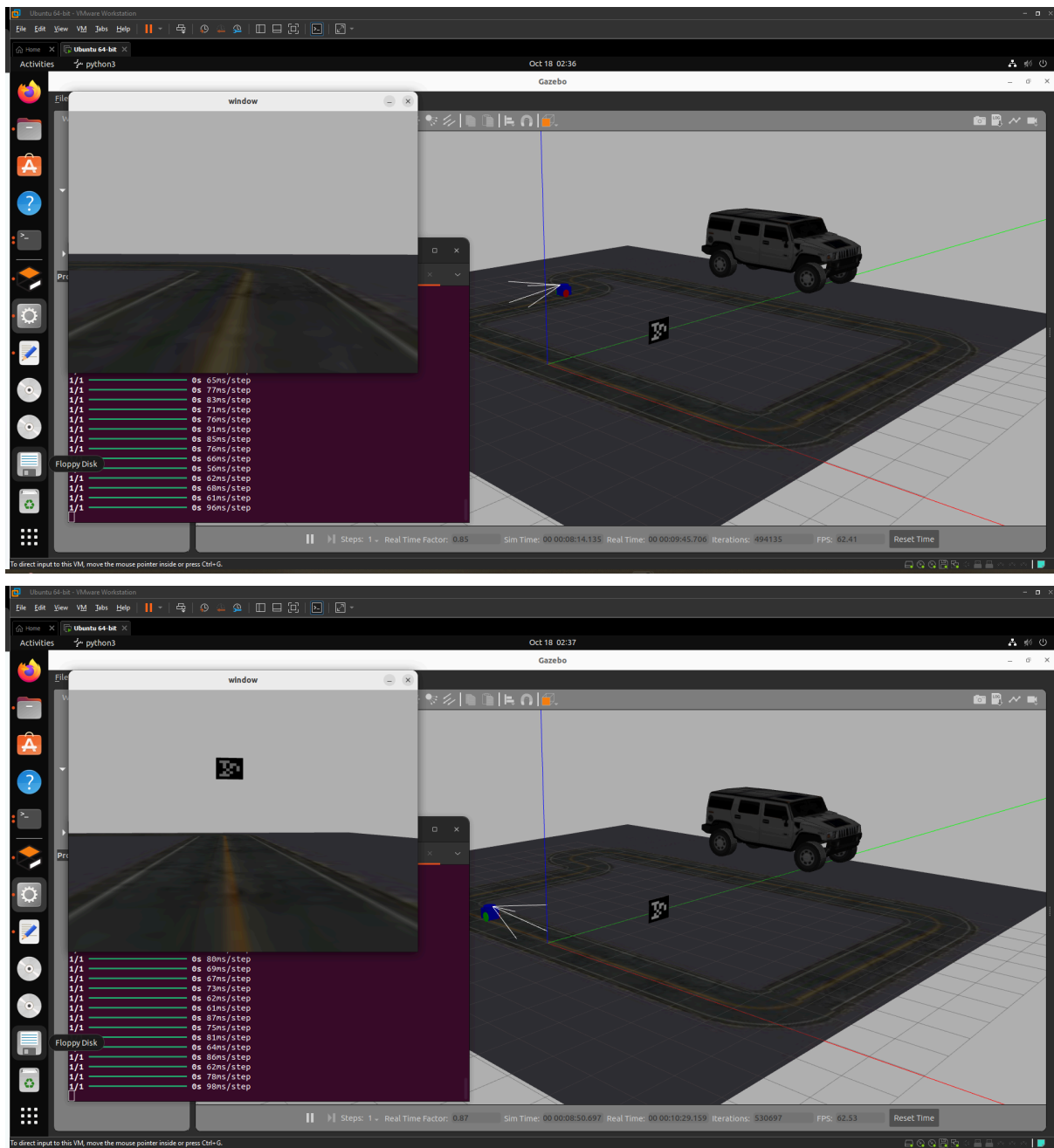
# save the model to disk
print("[INFO] serializing network...")
model.save("road-follower.keras")

```

road-follower-v3.py results:







road-follower-v3.py performance:

Overall best performance compared to original road-follower.py and road-follower-v2.py

Recap

	Performance	Errors	Overall
road-follower.py (original, pre-implemented without modifications)	Confident and stayed on path until the SUV	Jitters in proximity to objects and veers off course when next to suv and never is able to get back on track	Fail; cannot successfully complete course
road-follower-v2.py (adapted convolutional layers)	More Confident around objects as it did not veer off as much as the original when encountering the suv, but unable to manoeuvre correctly around them	When it would hit the turn it veered of the path quite a bit	Fail; cannot successfully complete course and wanders off course
road-follower-v3.py (with dropout layers)	Stays on track, confident around objects, best at following the centerline of the road	Moved around the vehicle placed on the track but then successfully manoeuvres back automatically	Success; Best performance, best at following the line and relentlessly completed

Supplementary Code

```
drive_by_road.launch.py:
import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument, IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node
import xacro

def generate_launch_description():
    urdf = os.path.join(get_package_share_directory('cpmr_ch6'), 'scout-camera.urdf.xacro')
    robot_desc = xacro.process_file(urdf, mappings={'name' : 'road_follower_robot'}).toxml()

    return LaunchDescription([
        IncludeLaunchDescription(
            PythonLaunchDescriptionSource(os.path.join(get_package_share_directory('gazebo_ros'),
                                                        'launch', 'gazebo.launch.py'),
            )
        ),
        Node(
            package='robot_state_publisher',
            executable='robot_state_publisher',
            name='robot_state_publisher',
            output='screen',
            parameters=[{'use_sim_time': False, 'robot_description': robot_desc}],
            arguments=[urdf]),
        Node(
            package='gazebo_ros',
            executable='spawn_entity.py',
            name='urdf_spawner',
            output='screen',
            arguments=["-topic", "/robot_description", "-entity", "road_follower_robot"]),
        Node(
            package='cpmr_ch6',
            executable='drive_by_road',
            name='drive_by_road',
            output='screen'),
    ])
```

```

auto_drive_by_road.launch.py:
import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument, IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node
import xacro

def generate_launch_description():
    urdf = os.path.join(get_package_share_directory('cpmr_ch6'), 'scout-camera.urdf.xacro')
    robot_desc = xacro.process_file(urdf, mappings={'name' : 'sonar_robot'}).toxml()

    return LaunchDescription([
        IncludeLaunchDescription(
            PythonLaunchDescriptionSource(os.path.join(get_package_share_directory('gazebo_ros'),
                                                        'launch', 'gazebo.launch.py'),
            ),
        ),

        Node(
            package='robot_state_publisher',
            executable='robot_state_publisher',
            name='robot_state_publisher',
            output='screen',
            parameters=[{'use_sim_time': False, 'robot_description': robot_desc}],
            arguments=[urdf]),

        Node(
            package='gazebo_ros',
            executable='spawn_entity.py',
            name='urdf_spawner',
            output='screen',
            arguments=["-topic", "/robot_description", "-entity", "camera_robot"]),

        Node(
            package='cpmr_ch6',
            executable='auto_drive_by_road',
            name='auto_drive_by_road',
            output='screen'),
    ])

```

```

road-follower-test.py:
from packaging import version
import tensorflow as tf
if version.parse(tf.__version__) < version.parse("2.9.0"):
    from keras.preprocessing.image import img_to_array
else:
    from tensorflow.keras.utils import img_to_array
from keras.models import load_model
from imutils import paths
import numpy as np
import cv2
import random

model = load_model("road-follower.keras")
dataset = './trainImages/'
imagePaths = sorted(list(paths.list_images(dataset)))

dirs = ["left", "forward", "right"]
for name in imagePaths:
    image = cv2.imread(name)
    cv2.imshow("Image", image)
    cv2.waitKey(3)
    im = img_to_array(image)
    im = np.array(im, dtype="float") / 255.0
    im = im.reshape(-1, 28, 28, 3)
    prediction = np.argmax(model.predict(im))
    if name.find(dirs[prediction]) < 0:
        print(f'Failure {name} direction is {dirs[prediction]}')

```

```

auto_drive_by_road.py:
import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2' # disable information messages
from packaging import version
import tensorflow as tf
if version.parse(tf.__version__) < version.parse("2.9.0"):
    from keras.preprocessing.image import img_to_array
else:
    from tensorflow.keras.utils import img_to_array
from keras.models import load_model
import math
import numpy as np
from keras.models import load_model
import cv2
from cv_bridge import CvBridge, CvBridgeError
import rclpy
from rclpy.node import Node
from rclpy.parameter import Parameter
from nav_msgs.msg import Odometry
from geometry_msgs.msg import Twist
from sensor_msgs.msg import Image
from keras.layers import TFSMLayer

class AutoDriveByLine(Node):
    def __init__(self):
        super().__init__('drive_by_line')
        self.get_logger().info(f'{self.get_name()} created')

        self.declare_parameter('image', "/mycamera/image_raw")
        self.declare_parameter('cmd', "/cmd_vel")
        self.declare_parameter('odom', "/odom")
        self.declare_parameter('rate', 10)
        self.declare_parameter('model', "road-follower.keras")
        self.declare_parameter('x_vel', 0.2)
        self.declare_parameter('theta_vel', 0.2)
        self.declare_parameter('image_size', 28)

        self._image_topic = self.get_parameter('image').get_parameter_value().string_value
        self._cmd_topic = self.get_parameter('cmd').get_parameter_value().string_value
        self._rate = self.get_parameter('rate').get_parameter_value().double_value
        self._model = load_model(self.get_parameter('model').get_parameter_value().string_value)
        self._x_vel = self.get_parameter('x_vel').get_parameter_value().double_value
        self._theta_vel = self.get_parameter('theta_vel').get_parameter_value().double_value
        self._image_size = self.get_parameter('image_size').get_parameter_value().integer_value

        self.create_subscription(Image, self._image_topic, self._image_callback, 1)
        self._pub = self.create_publisher(Twist, self._cmd_topic, 1)

```

```

self._bridge = CvBridge()
self._auto_driving = False

def _image_callback(self, msg):
    image = self._bridge.imgmsg_to_cv2(msg, "bgr8")
    cv2.imshow('window', image)
    key = cv2.waitKey(3)

    if self._auto_driving:
        if key == 125:
            self.get_logger().info(f"Auto driving ending")
            self._auto_driving = False
            image = cv2.resize(image, (self._image_size, self._image_size))
            im = img_to_array(image)
            im = np.array(im, dtype="float") / 255.0
            im = im.reshape(-1, self._image_size, self._image_size, 3)
            id = np.argmax(self._model.predict(im))
            if id == 0:
                self.turn_left()
            elif id == 1:
                self.go_straight()
            else:
                self.turn_right()
        else:
            if key == 106:
                self.turn_left()
            elif key == 107:
                self.go_straight()
            elif key == 108:
                self.turn_right()
            elif key == 32:
                self.stop()
            elif key == 113:
                self.get_logger().info(f"Closing node")
                exit(0)
            elif key == 120:
                self.get_logger().info(f"Auto driving starting")
                self._auto_driving = True

def _command(self, x_vel, theta_vel):
    twist = Twist()
    twist.linear.x = x_vel
    twist.angular.z = theta_vel
    self._pub.publish(twist)

def go_straight(self):

```

```

        self._command(self._x_vel, 0.0)

    def turn_left(self):
        self._command(self._x_vel, self._theta_vel)

    def turn_right(self):
        self._command(self._x_vel, -self._theta_vel)

    def stop(self):
        self._command(0.0, 0.0)

def main(args=None):
    rclpy.init(args=args)
    node = AutoDriveByLine()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

```

drive_by_road.py:
# write to output/X/ where X is one of forward, left, right
import math
import numpy as np
import rclpy
from rclpy.node import Node
from rclpy.parameter import Parameter
import cv2
from cv_bridge import CvBridge, CvBridgeError
from sensor_msgs.msg import Image
from nav_msgs.msg import Odometry
from geometry_msgs.msg import Twist

class DriveByRoad(Node):
    _FORWARD = 1
    _TURNING_LEFT = 2
    _TURNING_RIGHT = 3

    def __init__(self):
        super().__init__('drive_by_line')
        self.get_logger().info(f'{self.get_name()} created')

        self.declare_parameter('image', "/mycamera/image_raw")
        self.declare_parameter('cmd', "/cmd_vel")
        self.declare_parameter('odom', "/odom")
        self.declare_parameter('rate', 10)
        self.declare_parameter('output', "output")
        self.declare_parameter('x_vel', 0.2)
        self.declare_parameter('theta_vel', 0.2)
        self.declare_parameter('image_size', 28)

        self._image_topic = self.get_parameter('image').get_parameter_value().string_value
        self._cmd_topic = self.get_parameter('cmd').get_parameter_value().string_value
        self._rate = self.get_parameter('rate').get_parameter_value().double_value
        self._output = self.get_parameter('output').get_parameter_value().string_value
        self._x_vel = self.get_parameter('x_vel').get_parameter_value().double_value
        self._theta_vel = self.get_parameter('theta_vel').get_parameter_value().double_value
        self._image_size = self.get_parameter('image_size').get_parameter_value().integer_value

        self.create_subscription(Image, self._image_topic, self._image_callback, 1)
        self._pub = self.create_publisher(Twist, self._cmd_topic, 1)

        self._bridge = CvBridge()
        self._recording = False
        self._left_id = 0
        self._right_id = 0
        self._forward_id = 0

```

```

self._curdir = None

def _image_callback(self, msg):
    image = self._bridge.imgmsg_to_cv2(msg, "bgr8")
    cv2.imshow('window', image)
    key = cv2.waitKey(3)

    if key == 106:
        self.turn_left()
        self._curdir = DriveByRoad._TURNING_LEFT
    elif key == 107:
        self.go_straight()
        self._curdir = DriveByRoad._FORWARD
    elif key == 108:
        self.turn_right()
        self._curdir = DriveByRoad._TURNING_RIGHT
    elif key == 32:
        self.stop()
        self._curdir = None
        self._recording = False
        print(f"No recording when stopped")
    elif key == 115:
        self.get_logger().info(f"Start recording to {self._output}/X/ where X is one of forward, left,
right")
        self._recording = True
        self._curdir = DriveByRoad._FORWARD
        self.go_straight()
    elif key == 120:
        self.get_logger().info(f"Stop recording left {self._left_id} forward {self._forward_id} right
{self._right_id}")
        self._recording = False

    if self._recording:
        dim = min(image.shape[0], image.shape[1])
        oy = int((image.shape[0] - dim) / 2)
        ox = int((image.shape[1] - dim) / 2)
        roi = image[oy:oy+dim, ox:ox+dim, 0:3]
        out = cv2.resize(image, (self._image_size, self._image_size))
        cv2.imshow("Resized", out)
        if self._curdir == DriveByRoad._FORWARD:
            cv2.imwrite(f"{self._output}/forward/image.{self._forward_id:08d}.jpg", out)
            self._forward_id = self._forward_id + 1
        elif self._curdir == DriveByRoad._TURNING_LEFT:
            cv2.imwrite(f"{self._output}/left/image.{self._left_id:08d}.jpg", out)
            self._left_id = self._left_id + 1
        elif self._curdir == DriveByRoad._TURNING_RIGHT:
            cv2.imwrite(f"{self._output}/right/image.{self._right_id:08d}.jpg", out)
            self._right_id = self._right_id + 1

```



```

cv2.imshow('window', image)

def _command(self, x_vel, theta_vel):
    twist = Twist()
    twist.linear.x = x_vel
    twist.angular.z = theta_vel
    self._pub.publish(twist)

def go_straight(self):
    self._command(self._x_vel, 0.0)

def turn_left(self):
    self._command(self._x_vel, self._theta_vel)

def turn_right(self):
    self._command(self._x_vel, -self._theta_vel)

def stop(self):
    self._command(0.0, 0.0)

def main(args=None):
    rclpy.init(args=args)
    node = DriveByRoad()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```